EXPERIMENT - 70

1. Problem Statement

Write a recursive function to calculate the **digital root of a number**. Use a **static** variable to count the number of recursive calls.

Example: Digital root of 9875 is 2.
9875 => 9 + 8 + 7 + 5 = 29
29 => 2 + 9 = 11
11 => 1 + 1 = **2**

2. Algorithm

Main Program

Step 1: START
Step 2: Read n.
Step 3: Call digital_root(n) and display the result, then display callCount.
Step 4: STOP

Function digital_root(n)

Step 1: Increase the static variable callCount by 1.
Step 2: If n < 10, return n.
Step 3: Set sum = 0. While n > 0, add n % 10 to sum and set n = n / 10.
Step 4: Return digital_root(sum).

3. Test Case Table

#	Number	Expected Output	Actual Output	Result
1	9875	Digital root: 2 Number of recursive calls: 4	Digital root: 2 Number of recursive calls: 4	**PASS**
2	38	Digital root: 2 Number of recursive calls: 3	Digital root: 2 Number of recursive calls: 3	**PASS**
3	5	Digital root: 5 Number of recursive calls: 1	Digital root: 5 Number of recursive calls: 1	**PASS**
4	10	Digital root: 1 Number of recursive calls: 2	Digital root: 1 Number of recursive calls: 2	**PASS**
5	0	Digital root: 0 Number of recursive calls: 1	Digital root: 0 Number of recursive calls: 1	**PASS**
6	123456789	Digital root: 9 Number of recursive calls: 3	Digital root: 9 Number of recursive calls: 3	**PASS**

4. Source Code

digroot70.c

```
/* Experiment 70: digital root of a number using recursion,
   with a static variable counting the calls. */
#include <stdio.h>

static int callCount = 0;    /* increased on every call of digital_root() */

int digital_root(int n);

int main(void)
{
    int n;

    printf("Enter a number: ");
    scanf("%d", &n);

    printf("Digital root: %d\n", digital_root(n));
    printf("Number of recursive calls: %d\n", callCount);
    return 0;
}

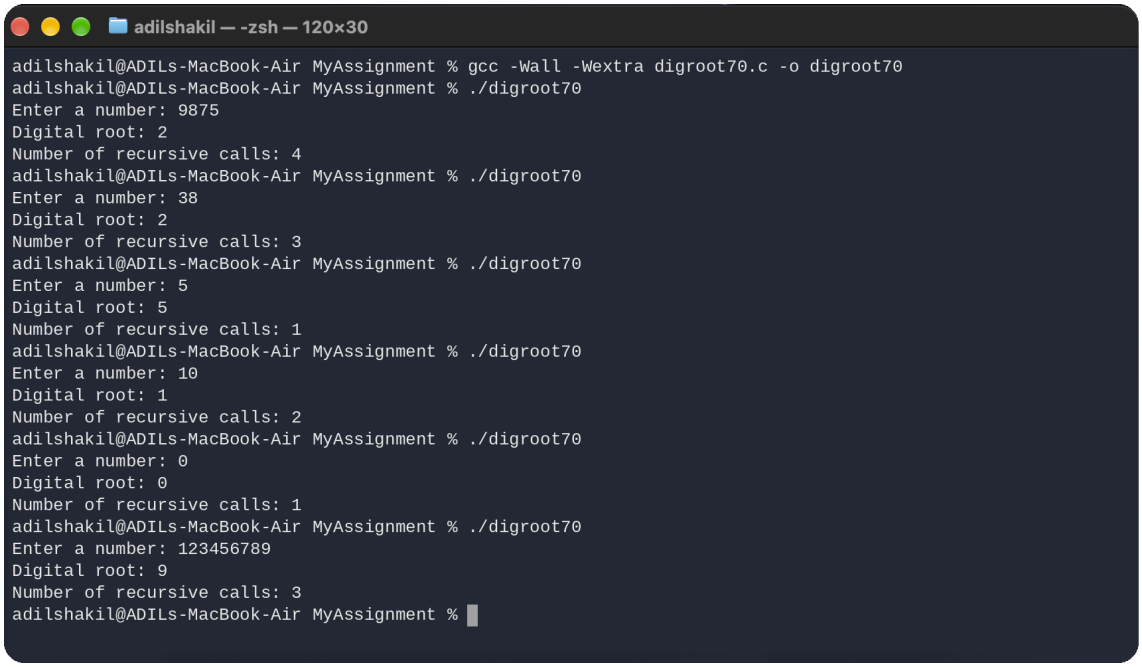
/* A single digit is its own digital root. Otherwise add up the digits
   and find the digital root of that sum, e.g. 9875 -> 29 -> 11 -> 2. */
int digital_root(int n)
{
    int sum = 0;

    callCount++;
    if (n < 10)
        return n;

    while (n > 0) {
        sum += n % 10;
        n /= 10;
    }
    return digital_root(sum);
}
```

5. Compilation and Output

Compiled with `gcc -Wall -Wextra digroot70.c -o digroot70` so the small warnings would show up too. It built clean.



All six passed.